

Linked List Confusion Notes

1. Fast & Slow Pointer Rule

If moving 1 step

```
curr = curr.next;
```

Use:

```
while(curr != null)
```

Examples:

- Reverse Linked List
 - Traverse List
 - Compare Two Lists
-

If moving 2 steps

```
fast = fast.next.next;
```

Use:

```
while(fast != null && fast.next != null)
```

Examples:

- Middle of Linked List
- Palindrome Linked List (finding middle)
- Cycle Detection

Reason:

```
fast.next.next
```

requires both:

```
fast != null  
fast.next != null
```

2. How to Decide Loop Condition

Using curr.val

```
curr.val
```

Use:

```
while(curr != null)
```

Using curr.next.val

```
curr.next.val
```

Use:

```
while(curr.next != null)
```

Using fast.next.next

```
fast.next.next
```

Use:

```
while(fast != null && fast.next != null)
```

3. Merge Two Sorted Lists

Wrong:

```
while(list1 != null || list2 != null)
```

Reason:

```
list1.val  
list2.val
```

need both lists alive.

Correct:

```
while(list1 != null && list2 != null)
```

Remaining nodes:

```
curr.next = list1;
```

or

```
curr.next = list2;
```

No loop needed.

Reason:

```
3 -> 5 -> 7
```

is already connected.

Attaching 3 automatically attaches:

```
3 -> 5 -> 7
```

4. Dummy Node Pattern

Used when:

- Delete head
- Insert before head
- Remove Nth Node
- Merge Lists
- Partition List

Pattern:

```
ListNode dummy = new ListNode(0);  
dummy.next = head;
```

Return:

```
return dummy.next;
```

5. Delete Node Pattern

Pattern:

```
while(curr.next != null)  
{  
    if(curr.next.val == val)  
    {  
        curr.next = curr.next.next;  
    }  
    else  
    {  
        curr = curr.next;  
    }  
}
```

6. Why if-else and not only if?

Use if-else

When one case should move and another should stay.

Example:

```
if(delete)
{
    stay;
}
else
{
    move;
}
```

Deleting:

```
curr.next = curr.next.next;
```

Stay on same node.

Reason:

```
1 -> 6 -> 6 -> 3
```

After deleting first 6:

```
1 -> 6 -> 3
```

Need to check second 6 again.

Use only if

When both cases continue normally.

Example:

```
if(arr[i] == target)
{
    count++;
}

i++;
```

Whether condition is true or false:

```
i++
```

must happen.

7. Linked List Interview Checklist

Before coding ask:

1. Can head be deleted? -> Use Dummy Node
 2. Am I using curr.val or curr.next.val? -> Choose correct loop condition
 3. Am I moving 1 step or 2 steps? -> Choose correct loop condition
 4. After deletion should pointer move? -> Usually NO
 5. Can consecutive nodes be deleted? -> Use if-else pattern
-

Golden Rule:

Whenever confused, ask:

"What object am I dereferencing?"

curr.val -> curr != null

curr.next.val -> curr.next != null

fast.next.next -> fast != null && fast.next != null